

Sensiq: An MCU-Based IoT Architecture for Environmental Monitoring

Technical Reports: CL-2026-42, selfref 2026

Department of Electrical Engineering, Media, and Computer Science
Ostbayerische Technische Hochschule Amberg-Weiden
Amberg, Germany

Abstract—Modulararbeit in Big Data, Cloud and NoSQL. Das ist der englische Abstract (Zusammenfassung). Nicht benennen: Modulararbeit, Personen, Studiengang, ... Sondern als Elevator Pitch

Index Terms—Cloud, Big Data, NoSQL, Modulararbeit

I. INTRODUCTION

Here we go. [robocuprescue2025]

Motivation, zusammenfassender Überblick des Anwendungskonzeptes, fachliches Alleinstellungsmerkmal (USP), Benennung von technischen Schlüssel-Bausteinen, Fachvokabular einführen.

II. RELATED WORK

andere Arbeiten oder Technologien (die als Inspiration dienen), Screenshots

III. REQUIREMENTS

Project Personas

To ensure the system meets the diverse needs of all stakeholders, the requirements are guided by the following personas. Each persona has a specific role, defined goals, and distinct interests in the system:

Chemist Christian Focuses on exact environmental conditions. His primary goal is to monitor precise laboratory temperatures to accurately plan experiments and evaluate temperature-dependent reaction rates without interference from climate fluctuations.

Occupational Safety Officer Olivia Prioritizes employee well-being. She requires immediate, clear indicators of air quality safety to proactively enforce protective measures and prevent workplace health hazards.

Facility Manager Martin Responsible for building operations and rapid incident response. He needs instant, unmissable alerts regarding critical sensor thresholds to take immediate corrective actions before physical risks escalate.

Project Manager Pia Looks at the big picture and long-term optimization. She aims to analyze historical data trends to identify recurring patterns, correlate them with facility operations, and implement targeted ventilation strategies.

A. US-01: Real-time Temperature Monitoring

As Chemist Christian, I want to see the exact current temperature in the laboratory, so that I can accurately plan my experiments and evaluate temperature-dependent reaction rates.

Acceptance Criteria:

AC-01 The dashboard displays the current temperature precisely in °C. Input: latest DB temperature readings. Output: exact numerical temperature value.

AC-02 The temperature value updates automatically when a new reading arrives — no manual reload required. Test: write new temperature values to DB; display must update within 10 s.

AC-03 Displayed temperature values are at most 30 s old. Test: compare value timestamps with system time; difference ≤ 30 s.

AC-04 If no current temperature data is available (gap > 30 s), a clearly readable notice “No current temperature data” is shown instead of an outdated value.

B. US-02: Workplace Air Quality Status

As Occupational Safety Officer Olivia, I want to see at a glance whether the air quality in the workspace is currently safe or critical, so that I can immediately initiate protective measures for the employees if necessary.

Acceptance Criteria:

AC-01 The dashboard shows the air quality as a colour-coded indicator based on safety thresholds: **Green** (safe), **Yellow** (elevated), **Red** (critical). Input: latest DB air quality readings. Output: correct status colour.

AC-02 The status colour updates automatically when a new reading arrives — no manual reload required. Test: write new air quality values to DB; display must update within 5 s.

AC-03 The displayed status is based on data that is at most 30 s old. Test: compare value timestamps with system time; difference ≤ 30 s.

AC-04 If no current air quality data is available (gap > 30 s), a clearly readable notice “No current air quality data” is shown instead of a status colour.

C. US-03: Critical Alert

As a facility manager Martin, I want an immediate, unmissable notification the second a sensor reading hits a critical limit, empowering me to take instant corrective action before the situation escalates into a physical health risk.

Acceptance Criteria:

- AC-01 An email notification is sent automatically within 10 s of a threshold exceedance.
- AC-02 The notification contains: measured value (incl. unit), measurement timestamp, and alert type (e.g. “CO critical”).
- AC-03 Consecutive exceedances of the same type within 5 min do not trigger a further email (debounce).
- AC-04 The notification is delivered even if the frontend is not open at trigger time.

D. US-03: 24-Hour History Chart

As Project Manager Pia, I want to analyze the sensor data from the past 24 hours in a clear historical chart, so that I can identify recurring patterns of poor air quality, correlate them with specific facility operations, and implement targeted ventilation strategies.

Acceptance Criteria:

- AC-01 Dashboard displays a line chart with readings from the last 24 hours.
- AC-02 Chart is fully interactive within 3 s of page load.
- AC-03 Periods without data are shown as visible line breaks — no silent interpolation.
- AC-04 Chart updates automatically when new readings arrive, without a page reload.

E. US-04: Custom History View

As Data Scientist Daniela, I want to extract historical sensor readings with a customizable time range and sampling resolution, so that I can generate precise, high-granularity datasets to train our predictive air-quality models.

Acceptance Criteria:

- AC-01 Dashboard provides date pickers for start/end and a resolution selector (hour/day).
- AC-02 A query for 30 days at daily resolution returns results in under 5 s.
- AC-03 If an end date before the start date is entered, a clear error message is shown and no query is executed.
- AC-04 Periods without data appear as an explicit “No data” row in the table and as a line break in the chart.

F. US-05: Sensor Status

As IT Administrator Ian, I want to instantly verify the connection status and the last heartbeat timestamp of each sensor, so that I can proactively detect offline devices and troubleshoot network dropouts before data gaps occur.

Acceptance Criteria:

- AC-01 Sensor status (“Active”/“Inactive”) is prominently visible. Status switches to “Inactive” when the last data point is more than 2 min ago.

- AC-02 Date and time of the last received reading are displayed next to the status (format: DD.MM.YYYY HH:MM:SS).
- AC-03 Status indicator updates automatically — no manual reload required.
- AC-04 If the database connection is temporarily unavailable, the notice “Connection interrupted” is shown instead of a false “Active” status.

G. Definition of Done

A user story is considered complete only when, in addition to its specific acceptance criteria, all of the following quality standards are met:

- **Code Review:** Reviewed and approved by at least one other developer (four-eyes principle).
- **Tests:** Unit tests written and passing; coverage $\geq 40\%$.
- **Documentation:** Technical docs and API interfaces updated where applicable.
- **Deployment:** Feature successfully deployed.
- **Functional Test:** Feature manually tested in staging without issues.

H. Prioritisation (MoSCoW)

Category	User Stories
Must Have	US-01 (Current Status Display), US-03 (Critical Alert),
Should Have	US-06 (Sensor Status)
Could Have	US-04 (24-Hour History Chart)
Won’t Have	US-05 (Custom History View — deferred to Release 2.

IV. MINIMUM VIABLE PRODUCT (MVP): IoT DATA INGESTION AND PROCESSING VIA AWS

A. Objective

The MVP implements a server-side IoT pipeline on Amazon Web Services (AWS). It focuses on ingesting simulated sensor data via MQTT, processing and storing it within the AWS ecosystem, and providing access through a Web API. This serves as the technical foundation for the system.

B. System Architecture

Data Generation & Transmission

Instead of physical hardware, a Python script simulates realistic sensor data. Messages are transmitted via the MQTT protocol to **AWS IoT Core**, ensuring a lightweight and software-defined testing environment.

Processing Pipeline

Incoming data is handled by an **AWS IoT Rule** which triggers a **Lambda function**. The function validates the payload and persists it in:

- **Amazon DynamoDB:** Primary storage for structured time-series data.
- **Amazon S3:** Long-term archival of raw data for cost-efficiency.

Web API Access

A REST API, built with **AWS API Gateway** and **Lambda**, provides two core endpoints:

- GET /live – Returns the most recent measurement.
- GET /history – Retrieves chronological data from DynamoDB.

C. Out of Scope

To focus on core infrastructure, the following elements are excluded from this project phase:

- Graphical Frontend/Dashboard for data visualization.
- Physical ESP32 hardware and real sensors.
- Automated user notifications (e.g., Push or Email alerts).

V. TECHNICAL CONCEPT

This section describes the overall technical architecture of the proposed system, including the embedded hardware, communication protocols, cloud infrastructure, data model, and planned extensions. The focus lies on the interaction of IoT devices with a scalable, serverless cloud backend for real-time and historical data processing.

A. Data Acquisition and Transmission

The local ESP32 microcontroller is equipped with multiple sensors for monitoring environmental conditions.

The DHT11 sensor measures temperature and humidity with a decent accuracy for indoor applications. This is complemented by the KY-028 thermistor for high precision temperature readings. The KY-026 flame sensor enables the detection of uncontrolled fire sources by measuring the intensity of infrared light emitted by flames [**ky026flame**].

An additional gas sensor (e.g., MQ135) must be determined based on the specific gases of interest, such as volatile organic compounds (VOCs), carbon monoxide, or others relevant to the use case.

The sensor data is read periodically, pre-processed on the ESP32, and encapsulated into a structured data object for transmission, as illustrated in ???. Each message contains device metadata (ID, location, uptime) and a nested structure for sensor measurements for further extensibility.

Messages are published to AWS IoT Core with the MQTT protocol and secured using TLS [**aws**]. Certificate-based authentication ensures device integrity and prevents unauthorized publishing to the cloud backend.

B. Cloud Processing Architecture

Incoming MQTT messages are processed within AWS using a serverless, event-driven architecture. The cloud backend is logically divided into two branches for data handling:

Live data is made available through a RESTful API endpoint GET /live, optimized for low-latency access to the most recent sensor values [**restful2000**]. Historical data is persisted for long-term storage and analysis, accessible via GET /history for in-depth time-series evaluation and trend analysis.

Additionally, an alerting mechanism is planned to be implemented using AWS SNS to notify users via email when certain

```
1 {
2   "deviceId": "esp32-lab-01",
3   "runningSince": "10000",
4   "location": "Lab A",
5   "measurements": {
6     "dht11_temperature": {
7       "value": 20,
8       "unit": "degree C"
9     },
10    "dht11_humidity": {
11      "value": 50,
12      "unit": "%"
13    },
14    "ky026_flame": {
15      "value": 4000,
16      "unit": "analog"
17    },
18    "ky028_termistor": {
19      "value": 22,
20      "unit": "degree C"
21    }
22  }
23 }
```

Listing 1. Example of the JSON data model for sensor value encapsulation on the ESP32.

thresholds are exceeded, such as high temperature. Future extensions may include the development of a React-based frontend dashboard for visualizing historical trends for specific time periods, enabling time-based filtering and comparison of environmental conditions.

C. Technology Stack

The following programming languages and technologies are utilized across the different components of the system:

- C++: ESP32 firmware for sensor data acquisition and MQTT communication.
- Python: Implementation of AWS Lambda functions for data processing and validation.
- TypeScript: Infrastructure definition and backend logic within AWS services.
- React: Web-based frontend for visualization and interactive data analysis.